Queue ADT, Operations & Array Implementation

Q. Define Queue as an Abstract Data Type (ADT). Explain its standard operations and demonstrate its implementation using an Array with a relevant code snippet.

Definition to Remember

*A **Que*
be remo

---

1. Queue as an ADT

As an Abstract Data Type, a Queue manages a collection of elements and restricts access to two specific ends:

* **Re

2. Standard Operations

| Operation | Description | Condition |

| :--- | :-

3. Array Implementation in C

Two pointers (`front` and `rear`) are initialized to `-1`.

```
#include <stdio.h>
#define

int queue[SIZE];
int front

void enqueue(int value) {
if (rear =


int dequeue() {
if (front
```

4. Advantages and Limitations

* **Advantage:** Array implementation is simple and fast (O(1) time complexity for enqueue/dequeue).

* **Ap

Must-Write Points (for 10 marks)

*1. Queu

Quick Recall

`Queue
-> Linea

Complete Executable C Code: Linear Queue Array Implementation

```
#include <stdio.h>

#define MAX 5

struct Queue {
    int item

void initQueue(struct Queue *q) {
    q->fron

int isFull(struct Queue *q) {
    return c

int isEmpty(struct Queue *q) {
    return c

void enqueue(struct Queue *q, int val) {
```

```
if (isFul

int dequeue(struct Queue *q) {
if (isEm

void display(struct Queue *q) {
if (isEm

int main() {
struct Q
enqueue(&q, 10);
enqueu
printf("Dequeued: %d\n", dequeue(&q));
display(
return 0;
}
```